

Solution Design Document

NutriTrack

Phase 3: RAG Nutrition Assistant Generative AI

Author: Abhijith Nallana

A self initiated personal project

Version 1.0 | July 2026

Contents

1. Version History
2. Executive Summary
3. Purpose of Solution Design Document
4. Project Background
5. Project Team
6. Point of Contact
7. Business Requirements
8. Scope
9. Success Criteria
10. Solution Overview
11. Automation Flow
12. Implementation Timeline
13. Technology Stack
14. Current State Process Flow
15. Future State Process Flow
16. Functional Requirements
17. Technical Requirements
18. Assumptions and Considerations
19. Risks and Dependencies
20. Appendix A. Retrieval and Calculation Examples

Version History

Version	Date	Author	Description
1.0	July 2026	Abhijith Nallana	Initial draft covering Phase 3: the knowledge base, TF-IDF retrieval, grounded generation with Groq, the meal calculator tool, and Streamlit integration.

Executive Summary

Phase 3 adds a generative AI assistant to NutriTrack that answers nutrition and diabetes questions grounded in the project's own data. The assistant follows the retrieval augmented generation pattern: it first retrieves the most relevant facts from a knowledge base built directly from the Phase 1 database, then passes those facts to a Groq hosted language model that writes a short, cited answer. A strict system prompt keeps the model grounded in the retrieved facts, allows a clearly labelled fallback to general nutrition knowledge when the data does not cover the question, forbids diagnosis, and adds a disclaimer.

The assistant also exposes a meal calculator. When a user lists foods and amounts, the model parses them into food and gram pairs and calls a Python tool that looks each food up in the USDA table and sums the nutrition. The arithmetic runs in Python, not in the model, so the totals are exact and traceable. If no Groq key is present the page degrades gracefully to a retrieval only mode, so it is always demonstrable. Phase 3 is educational and does not diagnose diabetes or replace medical care.

Purpose of Solution Design Document

The purpose of this document is to define the scope, design, and technical approach for the Phase 3 assistant so that it is clear, repeatable, and straightforward to evaluate. It records the business and functional requirements, the retrieval and generation design, the meal calculator, the flow of a question through the system, the assumptions that shape the design, and the risks that need to be managed.

Project Background

A plain chatbot can invent nutrition numbers and health claims, which would be worse than no assistant at all. NutriTrack already holds trustworthy USDA food measurements and NHANES diabetes insights, so the natural design is to ground every answer in that data and cite it. Retrieval augmented generation is the standard way to do this: it keeps the fluency of a language model while tying its answers to a known source.

Project Team

NutriTrack is a self initiated personal project, designed and built from end to end by a single developer. There is no wider team. The project owner carries every role listed below.

- Knowledge base and retrieval: building the fact corpus from the database and the TF-IDF retriever.
- Generation and tools: the grounded prompt, the Groq call, and the meal calculator tool.
- Integration: the Streamlit assistant page with chat, sources, and graceful fallback.

- Documentation: preparing this design document.

Point of Contact

- Project owner and developer: Abhijith Nallana.

Business Requirements

The solution is expected to meet the following needs:

- Answer nutrition and diabetes questions grounded in the NutriTrack data, with citations.
- Give clearly labelled general nutrition guidance when the data does not cover the question.
- Calculate the total nutrition of a described meal accurately from the USDA data.
- Stay safe: never diagnose, and remind users to see a professional for personal health.
- Remain demonstrable without an API key through a retrieval only fallback.

Scope

In scope for Phase 3 is building the knowledge base from the Phase 1 database, a TF-IDF retriever over that base, grounded generation with a Groq hosted model under a strict prompt, a meal calculator tool that the model can call, a Streamlit assistant page with chat history and a sources panel, and a retrieval only fallback when no key is present.

Out of scope for Phase 3 is a vector database or learned embeddings, persistence of chat history across sessions, per user conversations, fine tuning of the model, and knowledge drawn from the open web. The assistant answers from the NutriTrack data and general nutrition knowledge only.

Success Criteria

Phase 3 is considered successful when the following are true:

- The knowledge base is built entirely from the database, and its figures match the Phase 1 analysis.
- Retrieval returns relevant facts for a question, ranked by similarity.
- Answers are grounded in the retrieved facts and cite them, with a labelled general fallback.
- The meal calculator returns exact totals computed in Python from the USDA data.
- The page works with a key for full answers and without a key in retrieval only mode.

Solution Overview

The assistant is built from four small parts that live in a single module and are used by one Streamlit page. A corpus builder turns the database into a list of short facts. A retriever ranks those facts against a question. A generator sends the question and the top facts to the Groq model under a grounded prompt. A calculator tool, which the model can call, looks foods up in the USDA table and sums their nutrition in Python. The architecture is shown in Figure 1.

Automation Flow



Figure 1. Phase 3 system architecture.

A question flows through the system in a fixed order. The retriever scores every fact against the question and returns the top matches. Those facts are numbered and placed in a context block with the question. The Groq model answers from that block, and if the question is a meal it calls the calculator tool, which runs in Python and returns exact totals that the model then reports. The sources used are shown under the answer. Figure 2 shows this flow.



Figure 2. Phase 3 automation flow.

Implementation Timeline

- **Phase 1, the data pipeline, exploratory analysis, and Streamlit application:** complete.
- **Phase 2, the diabetes risk model and its risk assessment page:** complete.
- **Phase 3, the retrieval augmented nutrition assistant:** complete.

Technology Stack

- **Language:** Python.
- **Retrieval:** scikit learn TF-IDF vectoriser and cosine similarity.
- **Generation:** the Groq API, default model `openai/gpt-oss-20b`.
- **Storage:** SQLite, read from Phase 1.
- **Application:** Streamlit chat interface.

Current State Process Flow

Without grounding, a language model asked a nutrition question can produce a confident but invented number, and a user has no way to tell. There is no link to the trustworthy data that already exists in the project. Figure 3 illustrates this current state.



Figure 3. Current state process flow.

Future State Process Flow

In the future state the assistant retrieves facts from the NutriTrack data, answers from them and cites them, calculates meals with a Python tool, and falls back to labelled general guidance only when the data does not cover the question. Figure 4 shows this target flow.



Figure 4. Future state process flow.

Functional Requirements

- **Knowledge base:** build short facts from the database, covering USDA foods, NHANES insights, and reference facts.
- **Retrieval:** rank facts against a question with TF-IDF cosine similarity and return the top matches.
- **Grounded generation:** answer from the retrieved facts, cite them, and allow a labelled general fallback.
- **Meal calculator:** accept a described meal, look each food up in the USDA table, and return exact totals.
- **Sources display:** show the facts used under each answer.
- **Fallback:** present the retrieved facts directly when no API key is set.

Technical Requirements

- The corpus must be built from the Phase 1 database in a read only manner.
- Retrieval must use TF-IDF vectors and cosine similarity over the fact corpus.
- The Groq key must be read from the environment or from the Streamlit secrets file, and must never be committed.
- The meal arithmetic must run in Python against the foods table, not in the language model.
- The page must degrade to a retrieval only mode when no key is available.

Assumptions and Considerations

- The model estimates grams for counts and scoops, so portion sizes are approximate while the per 100 gram nutrition and the summing are exact.
- Food matching is to the 328 USDA foundation foods, so the calculator picks the closest available item and shows which one it used. Branded items may not be present.
- General nutrition knowledge used as a fallback is labelled as general guidance, not as data from NutriTrack.
- The assistant is for personal and educational use only and does not provide medical diagnosis or treatment advice.

Risks and Dependencies

- **Portion estimates:** grams for counted foods come from the model, so they are approximate.
- **API key and limits:** the assistant depends on a Groq key and its free tier rate limits.
- **Model availability:** Groq deprecates models over time, so the default model id is a single line that can be swapped for a current one from the Groq models endpoint.

- **Retrieval quality:** on a small corpus, similarity scores are modest, so the grounded prompt and the labelled fallback matter.

Appendix A. Retrieval and Calculation Examples

The examples below are produced by the Phase 3 module against the real database.

Knowledge base composition

The corpus holds 336 facts: 328 USDA food facts, 5 NHANES insight facts computed live from the survey table, and 3 reference facts, including the Phase 2 model card. The NHANES figures match the Phase 1 analysis, for example about 10 percent diabetes prevalence and a glucose correlation of 0.56.

Retrieval examples

For each question the retriever returns the most similar facts.

Question	Top retrieved source
How does BMI relate to diabetes?	NHANES insight: feature correlations with diabetes
How much protein is in dried egg white?	USDA food: Egg, white, dried
What features does the risk model use?	NutriTrack Phase 2 model card
At what fasting glucose is the diabetes range?	Reference: ADA fasting glucose thresholds

Meal calculation example

For the meal “5 almonds, 5 cashews, 1 scoop oats, 1 scoop protein powder, 1 banana, 350 ml milk”, the model converts each item to grams and the Python tool looks them up and sums them. The matched foods and totals are shown below. Protein powder is not in the USDA foundation set, so it is reported as not found and left out of the totals.

Ingredient (matched USDA food)	Grams	Calories	Protein (g)
almonds (Nuts, almonds, whole, raw)	6	35.0	1.3
cashews (Nuts, cashew nuts, raw)	8	42.7	1.4
oats (Oats, whole grain, steel cut)	40	151.7	5.0
banana (Bananas, raw)	118	100.3	0.9
milk (Milk, whole, 3.25% milkfat)	360	216.0	11.8
Total		545.7	20.4

Interpretation

The totals are computed in Python from the USDA data, so they are exact and can be traced to the matched foods. The language model only parses the meal and estimates portions; it does not do the arithmetic. This is the key design choice that keeps the numbers trustworthy.